

Simulating Ground States and Dynamics of Small Quantum Many-Body Systems Using DMRG

Junjie Xu, Jingyi Wang, Jiangteng Wang, Siyi Li

Zhejiang University

Outline

DMRG Algorithm Principles

How the density matrix renormalization group variationally optimizes a matrix product state.

TEBD vs DMRG

Comparing two core MPS algorithms: imaginary-time evolution vs. variational sweeping.

DMRG Applications

H₂ molecule and 1D hydrogen chain.

3.1 Single H₂ Molecule Ground State

Calculating bond length, energy, and electron correlation.

3.2 1-D Hydrogen Chain (H_{1D}): Energy & Geometry

Finding the optimal dimerized configuration and proving Peierls dimerization.

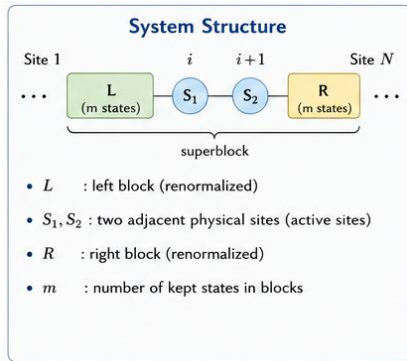
DMRG Algorithm Principles

1.1 Background and motivation

- Calculating **the ground states of quantum many-body systems** is a fundamental problem in condensed matter physics and quantum chemistry. For one-dimensional strongly correlated systems, **exact diagonalization (ED)** can provide benchmark results, but its computational complexity grows exponentially with system size, severely limiting the size of systems that can be studied.
- Since its introduction by **Steven R. White** in 1992, **the Density Matrix Renormalization Group (DMRG)** has become one of the most important numerical tools for solving one-dimensional strongly correlated systems. White's groundbreaking work shifted the approach to quantum many-body problems from traditional "diagonalizing large matrices" to **"constructing and optimizing the reduced density matrix between the system and the environment."**

1.2 Density Matrix Renormalization Group

- **Decomposition** (块分解) : Decompose the many-body wavefunction on a 1D lattice into a small "active" system and large "renormalized" environments.
- **Truncation** (截断) : Retain only the most significant states of the environments by diagonalizing the reduced density matrix.
- **Variational Optimization** (变分优化) : Iteratively improve the wavefunction by sweeping the active part across the lattice back and forth until convergence.



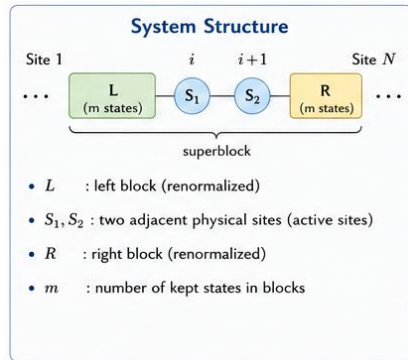
1.2 Density Matrix Renormalization Group

Goal: **Reduce the dimension** of blocks L and R while **preserving accuracy**.

$$\mathcal{H} = \mathcal{H}_L \otimes \mathcal{H}_{S_1} \otimes \mathcal{H}_{S_2} \otimes \mathcal{H}_R$$

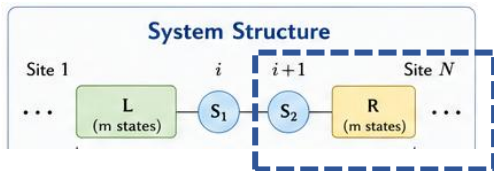
$$|\Psi\rangle = \sum_{\alpha, i, j, \beta} \Psi_{\alpha i j \beta} |\alpha\rangle_L |i\rangle_{S_1} |j\rangle_{S_2} |\beta\rangle_R$$

where $\Psi_{\alpha i j \beta}$ are the **probability amplitudes**



1.2 Density Matrix Renormalization Group

Goal: Reduce the dimension of blocks L and R while preserving accuracy.



Truncation error

$$\epsilon = 1 - \sum_{i=1}^m \omega_i$$

Controlled by m .

Method

1. Construct the reduced density matrix for block $L-S_1$:

$$\rho_{LS_1} = \text{Tr}_{S_2R} (|\Psi\rangle\langle\Psi|)$$

2. **Diagonalize** ρ_{LS_1} :

$$\rho_{LS_1} |\phi_k\rangle = w_k |\phi_k\rangle$$

where w_k are eigenvalues, $w_1 \geq w_2 \geq \dots$

3. **Truncation:** Keep only the m most important states with largest w_k :

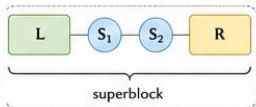
$$|\Psi\rangle \approx \sum_{k=1}^m \sqrt{w_k} |\phi_k\rangle_{LS_1} \otimes |\chi_k\rangle_{S_2R}$$

1.3 DMRG Algorithm

Sweeping: Update $L-S_1-S_2-R$

Left-to-Right Sweep ($i = 1 \rightarrow N-1$)

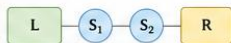
① Build the superblock



Form the superblock Hamiltonian

$$H_{\text{super}} = H_L + H_{S_1} + H_{S_2} + H_R \\ + H_{LS_1} + H_{S_1S_2} + H_{S_2R}$$

② Solve for ground state



Find the ground state $|\Psi\rangle$ of H_{super} by Lanczos or Davidson.

③ Update L (grow left block)



Form the reduced density matrix for the left part (L and S_1):

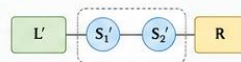
$$\rho_{LS_1} = \text{Tr}_R |\Psi\rangle\langle\Psi|$$

Keep the m eigenstates with the largest eigenvalues.

Define the new left block:

$$L' = \text{Renorm}(L, S_1) \quad (\text{dim} = m)$$

④ Move the center to the right



Set $i \leftarrow i + 1$.

The new system is $L' - S_1' - S_2' - R$.

Repeat from ① until $i = N - 1$.

1.4 Algorithm Implementation

We study **the one-dimensional XXZ model** with open boundary conditions:

$$H = \sum_i (S_i^x S_{i+1}^x + S_i^y S_{i+1}^y + \Delta S_i^z S_{i+1}^z)$$

- L : number of lattice sites
- S_i^α : spin-1/2 operators at site i
- Δ : anisotropy parameter

Goal: Find the **ground state energy** E_0 and **wavefunction** $|\Psi_0\rangle$

1.4 Algorithm Implementation

Step 1 & 2: 构建超块并求解基态

```
H_super = self.build_superblock(left, right)
eigvals, eigvecs = eigh(H_super)
energy = eigvals[0].real
psi = eigvecs[:, 0]
```

Step 3: 计算约化密度矩阵

```
dim_l, dim_r = left['dim'], right['dim']
psi_mat = psi.reshape(dim_l, dim_r)
rho = psi_mat @ psi_mat.conj().T
rho /= np.trace(rho)
```

Step 4: 对角化并截断

```
rho_eigvals, rho_eigvecs = eigh(rho)
idx = np.argsort(rho_eigvals)[::-1]
chi = min(self.chi_max, len(rho_eigvals))
proj = rho_eigvecs[:, idx[:chi]]
```

初始化: 两个单格点

```
left = self.init_site()
right = self.init_site()
```

逐步扩增到目标大小

```
for step in range(1, self.L // 2):
    left = self.enlarge(left)
    right = self.enlarge(right)

    left, energy, err_l = self.step(left, right)
    right, energy, err_r = self.step(right, left)

    current_size = 2 * (step + 1)
```

1.4 Algorithm Implementation

阶段 1: 无限 DMRG (系统逐步生长)

Step	系统大小	保留维度	能量	截断误差
1	4	4	能量: -1.61602540, 维度: 4, 截断误差: 3.33e-16	
			能量: -1.61602540, 维度: 4, 截断误差: -2.22e-16	
			-1.61602540	3.33e-16
			能量: -2.49357713, 维度: 8, 截断误差: 3.33e-16	
2	6	8	能量: -2.49357713, 维度: 8, 截断误差: -2.22e-16	
			-2.49357713	3.33e-16
			能量: -3.37493260, 维度: 16, 截断误差: 1.33e-15	
			能量: -3.37493260, 维度: 16, 截断误差: 1.11e-15	
3	8	16	-3.37493260	1.33e-15

✓ 达到目标系统大小 8

基态能量:

总能量 $E_{\text{total}} = -3.3749325987$

每格点能量 $E/L = -0.4218665748$

参考值 (热力学极限): -0.4431

有限尺寸效应: 0.0212

计算时间: 0.08 秒

1.5 Modern DMRG Improvement: SVD

- Modern DMRG (especially within the **MPS** framework) directly performs **SVD** on the wavefunction matrix. **The two approaches are mathematically equivalent.**

Let Ψ be the ground state wavefunction reshaped into a matrix:

Traditional approach Construct left-block density matrix $\rho_L = \Psi\Psi^\dagger$
then diagonalize:

$$\rho_L = U\Lambda U^\dagger$$

Modern approach (SVD)

Directly perform SVD on the wavefunction:

$$\Psi = USV^\dagger$$

Substitute the SVD expression into the RDM formula: $\rho_L = (USV^\dagger)(VSU^\dagger) = US^2U^\dagger$

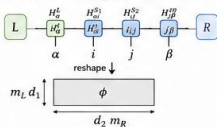
1.5 Modern DMRG Improvement: SVD

- Modern DMRG (especially within the **MPS** framework) directly performs **SVD** on the wavefunction matrix. **The two approaches are mathematically equivalent.**
- Directly perform SVD on the wavefunction: $\Psi = USV^\dagger$

Sweeping update (one step) — optimize S_1, S_2 and update via SVD of ϕ

1 Build $|\phi\rangle$

Contract L, S_1, S_2, R to form $\phi_{(\alpha i)(j\beta)}$ by grouping indices (α, i) as rows and (j, β) as columns.



2 Local eigenproblem

Solve the effective Schrödinger equation in this subspace (using Davidson/Lanczos):

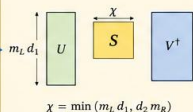
$$H_{\text{eff}} |\phi_k\rangle = E_k |\phi_k\rangle$$

Get the ground state $|\phi_0\rangle$ and reshape to ϕ_0

3 SVD of ϕ_0 (modern DMRG)

Perform SVD directly on ϕ_0 :

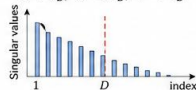
$$\phi_0 = USV^\dagger$$



4 Truncate

Keep the largest D singular values (retain states with largest weight):

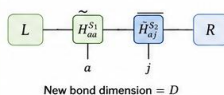
$$S \rightarrow S_D, U \rightarrow U_D, V \rightarrow V_D$$



$$\text{Truncation error: } \epsilon = \sqrt{\sum_{k>D} s_k^2}$$

5 Update tensors and move center

Absorb U_D and S_D into the left tensor, and set the right tensor to $V_D^\dagger$.



Sweep directions

left $\rightarrow$ right $\rightarrow$ left $\rightarrow$ right ...
(1 sweep = one full left-to-right and right-to-left pass)

Environments update

After updating the tensors, recompute L or R environments incrementally (see below).

Convergence

Iterate sweeps until energy change and truncation error are below thresholds.

TEBD vs DMRG

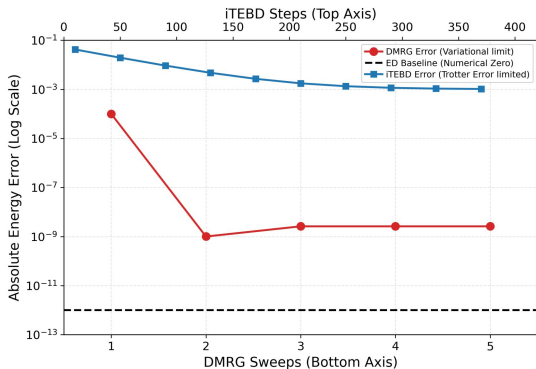
2.1 Time-Evolving Block Decimation

- **TEBD** is a tensor network algorithm for simulating **real / imaginary-time** evolution of 1D quantum systems. It represents the quantum state as a Matrix Product State (**MPS**) and updates local tensors using singular value decomposition (**SVD**). It is ideal for **finite-size systems** and **short-time dynamics**.
- **iTEBD** extends TEBD to **infinite-size systems** by assuming a uniform translational invariant MPS. It works directly in the thermodynamic limit using a unit cell representation. iTEBD is efficient for finding ground states of infinite systems and for studying **quantum quenches** in the thermodynamic limit.

2.2 Comparative Study of DMRG and iTEBD

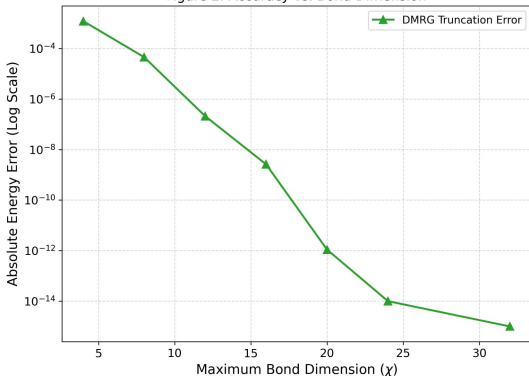
Ground State Energy Convergence (収斂性)

Figure 1: Ground State Energy Convergence Error



Accuracy vs. Bond Dimension

Figure 2: Accuracy vs. Bond Dimension



2.3 DMRG Core Advantages : Static Ground State Solving

1 No Trotter Error

- iTEBD relies on the Suzuki-Trotter decomposition of the time evolution operator, which inevitably introduces a **systematic error** proportional to the time step dt . DMRG completely bypasses time evolution, reformulating the problem as a **spatial variational optimization**. For a given bond dimension χ , DMRG searches for the **absolute minimum** of the energy within the variational manifold.

```
=== Method Comparison: Ground State Energy ===  
ED Energy: -4.2580352073  
iTEBD Energy: -4.2569375212 | Relative Error: 2.5779e-04  
DMRG Energy: -4.2580352046 | Relative Error: 6.2643e-10
```

2 Fast Convergence & High Precision

- DMRG typically reaches machine-precision accuracy within just a few sweeps.

Computation of H-Chain

Goals

- Use DMRG to find H₂'s ground state energy, to verify its accuracy.
- Find the optimal bond length for 1D equal-spacing H-chain and their ground state energy. Then Compare with independent H atoms to show the thermodynamic stability.
- Further analysis about the stability of 1D H-Chain.

Computational Methodology

- Main modules we use:
 - PySCF - RHF molecular orbitals
 - block2 - matrix product states (MPS) and DMRG
- Workflow:
 - Build molecule using coordinates and STO-3G/STO-6G basis set
 - Get 1e/2e integrals (h1e, g2e) & core energy
 - Initialize DMRG driver, MPO and random MPS
 - Sweep with increasing bond dimension
 - Converge energy & extract optimal geometry

Workflow example - H_2

```
mol = gto.M(atom="H 0 0 0; H 0 0 0.74", basis="sto3g", symmetry="d2h", verbose=0)
mf = scf.RHF(mol).run(conv_tol=1e-12)
```

- Molecule building:
 - 2 hydrogen atoms at (0,0,0) and (0,0,0.74)
 - using STO-3G basis set (a minimal basis set, fast but not very accurate)
- RHF(Restricted Hartree-Fock):
 - compute the initial molecule orbit, which is a simplified model that DMRG will start from

Workflow example - H_2 - Cont.

- Setup Hamiltonian:

Hamiltonian of H_2 is:

$$H = -\frac{1}{2}\nabla_1^2 - \frac{1}{2}\nabla_2^2 - \frac{1}{|\mathbf{r}_1 - \mathbf{R}_a|} - \frac{1}{|\mathbf{r}_1 - \mathbf{R}_b|} - \frac{1}{|\mathbf{r}_2 - \mathbf{R}_a|} - \frac{1}{|\mathbf{r}_2 - \mathbf{R}_b|} + \frac{1}{|\mathbf{r}_1 - \mathbf{r}_2|} + \frac{1}{|\mathbf{R}_a - \mathbf{R}_b|}$$

- DMRG cannot directly process the differential equations in continuous space, need to project H onto the molecular orbitals calculated by RHF

One-electron integrals h1e:

$$h_{pq} = \int \psi_p^*(\mathbf{r}) \left[-\frac{1}{2}\nabla^2 - \frac{1}{|\mathbf{r} - \mathbf{R}_a|} - \frac{1}{|\mathbf{r} - \mathbf{R}_b|} \right] \psi_q(\mathbf{r}) d\mathbf{r}$$

and Two-electron integral g2e:

$$g_{pqrs} = \iint \psi_p^*(\mathbf{r}_1) \psi_q(\mathbf{r}_1) \frac{1}{|\mathbf{r}_1 - \mathbf{r}_2|} \psi_r^*(\mathbf{r}_2) \psi_s(\mathbf{r}_2) d\mathbf{r}_1 d\mathbf{r}_2$$

Workflow example - H_2 - Cont.

- Finally, we use these operators to write the Hamiltonian in a discrete form, and this process is called the **second quantization** of the Hamiltonian.

$$\hat{H} = \sum_{pq} h_{pq} \hat{a}_p^\dagger \hat{a}_q + \frac{1}{2} \sum_{pqrs} g_{pqrs} \hat{a}_p^\dagger \hat{a}_r^\dagger \hat{a}_s \hat{a}_q + E_{core}$$

- The process mentioned above is done through:

```
ncas, n_elec, spin, ecore, h1e, g2e, orb_sym = itg.get_rhf_integrals(mf, ncore=0, ncas=None, g2e_symm=8)
```

- The variables are:

Number of orbitals (ncas): 2

Number of electrons (n_elec): 2

Total spin (spin): 0

Nuclear repulsion energy, core energy (ecore): 0.7151043391 Hartree

Workflow example - H_2 - Cont.

- Initialize DMRG: Specify symmetry and other parameters to boost calculation.

```
driver = DMRGDriver(scratch="./tmp", symm_type=SymmetryTypes.SU2, n_threads=4)
driver.initialize_system(n_sites=ncas, n_elec=n_elec, spin=spin, orb_sym=orb_sym)
mpo = driver.get_qc_mpo(h1e=h1e, g2e=g2e, ecore=ecore, iprint=0)
ket = driver.get_random_mps(tag="GS", bond_dim=20, nroots=1)
```

- Set DMRG sweep configurations:
 - bond_dims : 2 is enough for H_2 , 100 is an absolutely safe value.
 - noises : add noise to jump out of local minimum.
 - thrds : cut off thresholds for DMRG sweeps.

```
n_sweeps = 8
bond_dims = [100] * n_sweeps
noises     = [1e-4, 1e-5, 1e-6, 1e-7, 0, 0, 0, 0]
thrds      = [1e-10] * n_sweeps
```

Workflow example - H_2 - Cont.

- Run DMRG and output energy:

```
energy_DMRG = driver.dmrg(mpo, ket, n_sweeps=n_sweeps,  
                           bond_dims=bond_dims, noises=noises, thrds=thrds, iprint=0)  
print(f"H2 ground state energy = {energy_DMRG:.12f} Hartree")
```

- The result:

H2 ground state energy = -1.137283834489 Hartree

Benchmark on H_2 Molecule

- Bond length = 0.74Å, using STO-3G basis set
ground state energy = -1.137283834489 Hartree
- Compare with exact energy from PySCF.FCI :

```
fci_e = fci.direct_spin1.kernel(h1e, g2e, ncas, n_elec)[0] + ecore
print(f"PySCF FCI参考值: {fci_e:.12f} Hartree")
print(f"与DMRG计算结果的误差: {energy_DMRG - fci_e:.2e} Hartree")
```

- FCI reference value : -1.137283834489 Hartree
- The error is : -8.88e-16 Hartree
- The result of DMRG approach is consistent with FCI result.
- But is it close enough to real world?

Benchmark on H_2 Molecule - Cont.

- The ground state energy of H_2 is -1.1745 Hartree
- Our result is - 1.137 Hartree, still has a notable error.
- The error partly comes from the STO-3G basis - simple but inaccurate
- Change the basis to cc-pVTZ - a basis set including polarization functions

```
mol = gto.M(atom="H 0 0 0; H 0 0 0.74", basis="cc-pVTZ", symmetry=True, verbose=0)
```

- We get the result as -1.156572416521 Hartree, which is lower and also closer to the CBS(Complete Basis Set) limit.
- Besides, the DMRG intermediate results show that the accurate result is achieved just after the 1st sweep, the H_2 case is still too simple for it.

Analysis on Equal-Spacing H_n Chains

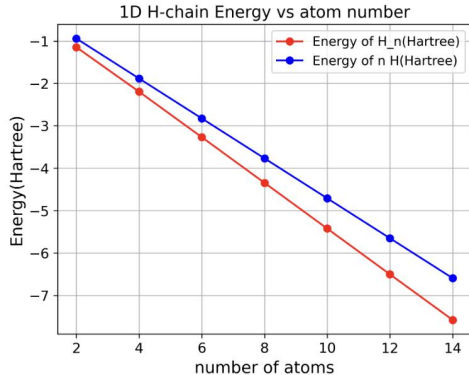
- We need to compute energy of H-chain and compare it with independent H atoms to show the thermodynamic stability.
- We set up equal-spacing H-chain with N atoms (N=2,4,6,8,10,12,14).
- For each H-chain, scan the bond length in range and run DMRG to find the minimal ground state energy.
- The molecule definition setup is done by:

```
def build_coords(n_atoms, bond_length):  
    coords = []  
    x = 0.0  
    for i in range(n_atoms):  
        coords.append(['H', (x, 0, 0)])  
        x += bond_length  
    return coords
```

```
coords = build_coords(n_atoms=n_atoms, bond_length=bond_length_angstrom)  
  
mol = gto.M(atom=coords, basis='sto6g', symmetry='c1', verbose=0)  
mf = scf.RHF(mol).run(conv_tol=1e-12)
```

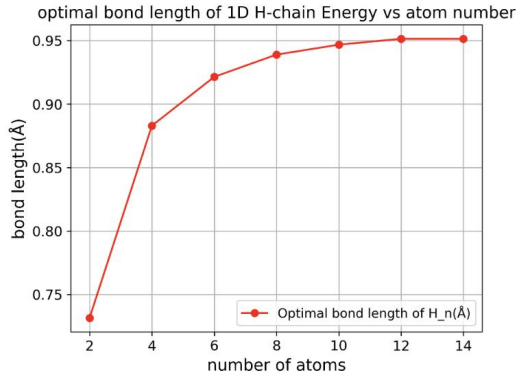
Analysis on Equal-Spacing H_n Chains - Cont.

- For all H-chains, we plot the energy v.s. atom number and optimal bond length v.s. atom number as following (with bond length $\epsilon = 0.02\text{\AA}$):



- For all H-chains, the energy is lower than the sum of the same number of independent H atoms, which means they are stable in thermodynamics.

Analysis on Equal-Spacing H_n Chains - Cont.



- The optimal bond length grows with the growing number of atoms, showing that the strength of bond is weakening as the chain grows. Especially for $N=2$ and $N=4$.

- But is equal-spacing ideal enough for 1D H-chains?
 - The drastic changes in energy around $N=2,4$ suggest that the H-chains might lower its total energy by forming units with smaller spacing.

Peierls Dimerization

- Existing studies on **Peierls dimerization** show that for one-dimensional metals, the system spontaneously shifts from a higher-energy 'metallic state' (with **uniform spacing**) to a lower-energy 'insulating state' (a dimerized structure with **alternating long and short bonds**).
- Whether dimerization occurs depends on the interplay of two types of energy:
 - electronic energy: The band gap opens, and the occupied state energy decreases.
 - elastic energy: Deviation from the equilibrium bond length, cost of lattice distortion.
- Total energy:
$$\Delta E(\delta) = \underbrace{A\delta^2}_{\text{Elastic penalty}} - \underbrace{B\delta^2 \ln \frac{1}{\delta}}_{\text{Electronic gain}}$$
- For small δ , we have large $\ln \frac{1}{\delta}$ and $\Delta E < 0$, so any one-dimensional half-filled metal will inevitably dimerize at zero temperature.

Considering Dimerized H_{10} Configuration

- In a 1D H-chain, each hydrogen atom contributes one electron, exactly filling half of the energy band, forming a 'half-full' **metallic state**. So lower energy is feasible through dimerization.

- Building dimerized molecule:

```
def build_dimerized_coords(n_atoms, R_short, R_long):  
    coords = []  
    x = 0.0  
    for i in range(n_atoms):  
        coords.append(['H', (x, 0, 0)])  
        if i % 2 == 0:  
            x += R_short  
        else:  
            x += R_long  
    return coords
```

Coarse Initial Scan (初始粗扫描)

```
n_atoms = 10  
R_short_range = np.arange(0.6, 1.3, 0.1) # 短键: 0.6 ~ 1.2 Å  
R_long_range = np.arange(0.8, 1.5, 0.1) # 长键: 0.8 ~ 1.4 Å
```

```
results = []  
for R_s in R_short_range:  
    for R_l in R_long_range:  
        if R_l <= R_s:  
            continue # 跳过无意义的组合 (长键必须大于短键)  
        try:  
            energy, e_rhf = compute_dimerized_energy(  
                R_s, R_l, n_atoms=10, basis='sto6g', verbose=0  
            )  
            results.append({  
                'R_short': R_s, 'R_long': R_l,  
                'E_DMRG': energy, 'E_RHF': e_rhf,  
                'E_corr': energy - e_rhf  
            })  
            print(f"Rs={R_s:.1f} Å, Rl={R_l:.1f} Å → E_DMRG = {energy:.10f} Hartree")  
        except Exception as e:  
            print(f"Rs={R_s:.1f}, Rl={R_l:.1f} 计算出错: {e}")
```

Conclution: Considering the Dimerized Configuration Calculation for H_{10}

H_{10} Peierls 二聚化扫描结果 (粗精度)

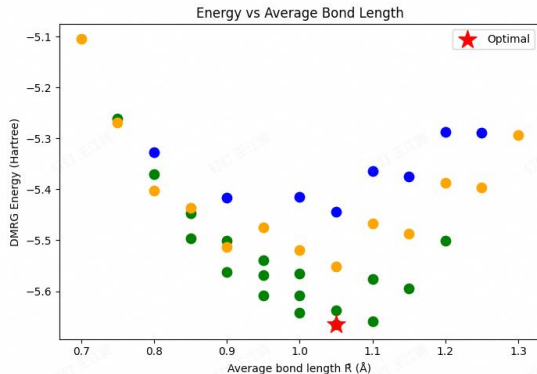
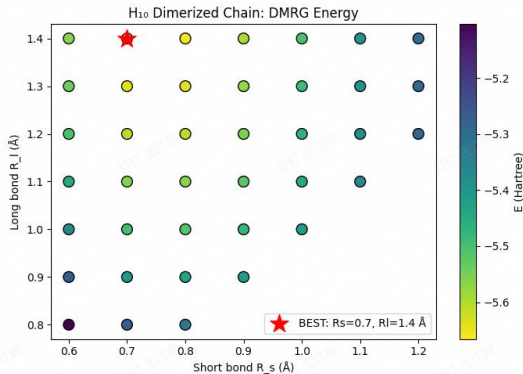
最优短键长: $R_{s}^* = 0.70 \text{ \AA}$

最优长键长: $R_{l}^* = 1.40 \text{ \AA}$

平均键长: $\bar{R} = 1.05 \text{ \AA}$

最低能量: $E_{\text{dim}} = -5.6656960951 \text{ Hartree}$

二聚化参数: $\delta \equiv (R_l - R_s)/\bar{R} = 0.667$



H₁₀ Peierls 二聚化扫描结果 (粗精度)

最优短键长: $R_{s}^* = 0.70 \text{ \AA}$

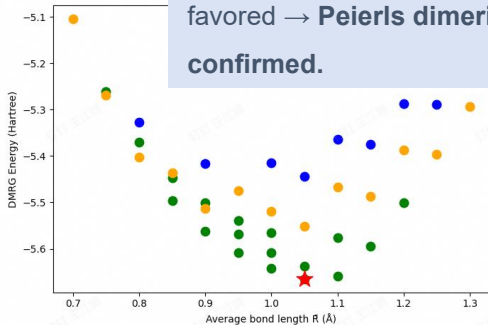
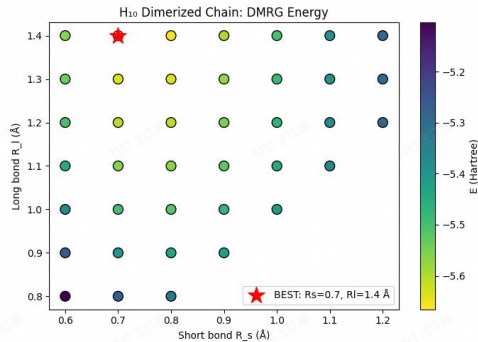
最优长键长: $R_{l}^* = 1.40 \text{ \AA}$

平均键长: $\bar{R} = 1.05 \text{ \AA}$

最低能量: $E_{\text{dim}} = -5.6656960951 \text{ Hartree}$

二聚化参数: $\delta \equiv (R_l - R_s)/\bar{R} = 0.667$

- Minimum energy ≈ -5.67 Hartree – lower than the uniform chain (all bonds equal).
- Diagonal positions (equal bond lengths) show higher energy.



So uniform distribution is not energetically favored → **Peierls dimerization effect is confirmed.**

Conclusion for H_{10}

- If we compare R_s with the bond length of H_2 , which is about $0.74\text{\AA}$, we'll find that the structure of dimerized H_{10} -chain looks like 5 hydrogen molecules.
- Energy of 5 H_2 molecules: -5.6864191724 Hartree
- Energy of H_{10} -chain: -5.6879675956 Hartree
- Error: -0.0015484231 Hartree (0.0272%)
- We can conclude that H_{10} -chain is in fact 5 H_2 molecules.

Main Conclusions

- The bond length of a 1D equally spaced H-chain increases as the number of atoms goes up, and the bonding gets weaker.
- Although a 1D equally spaced H-chain can be thermodynamically stable, in reality it's just metastable and tends to dimerize.
- Calculations for H_{10} show that after dimerization, the 1D H-chain is basically just independent H_2 molecules, and long one-dimensional hydrogen chains can't really exist stably.

Simulating Ground States and Dynamics of Small Quantum Many-Body Systems Using DMRG

Thank you for your attention.